

2025 Jan Oppe 1 - Set 2

Section 3 - problem 1

Employee Data Analysis

PROBLEM STATEMENT

Implement four functions for employee data (list of dictionaries):

1. List employees with salary $\geq$ minimum.
2. Calculate total salary for a department.
3. Round a number up to the next 500.
4. Find max salary in a department after a percentage increment (rounded up).



EXAMPLE & EXPLANATION

Sample Data: `[{'name': 'Alice', 'salary': 50000}, {'name': 'Bob', 'salary': 70000}]`

1. Employees with salary $\geq$ 60000: `['Bob', 'David']`
2. Total salary for 'HR': 95000
3. Round 25100 up to next 500: 25500
4. Max salary in 'Engineering' after 7% increment: 75000

NOTE: This is a function type question, you don't have to take input or print the output, just have to complete the required function definition.

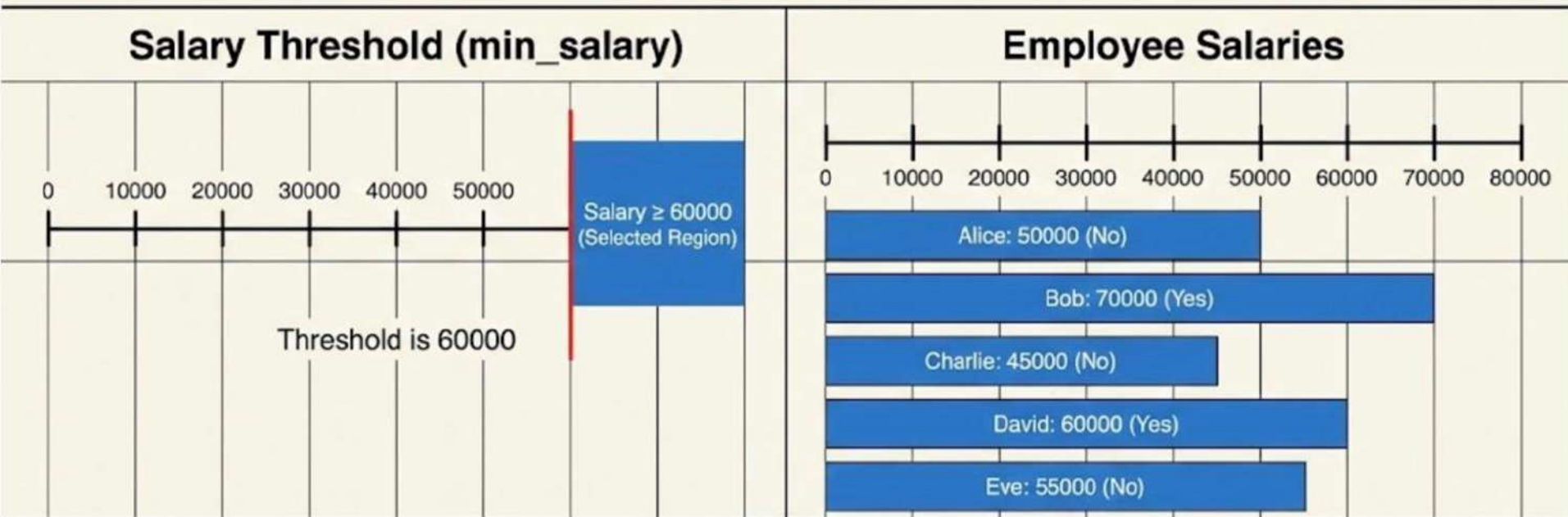
THE PROBLEM SPECIFICATION



— **Constraint:** This is a function type question, you don't have to take input or print the output, just have to complete the required function definition.

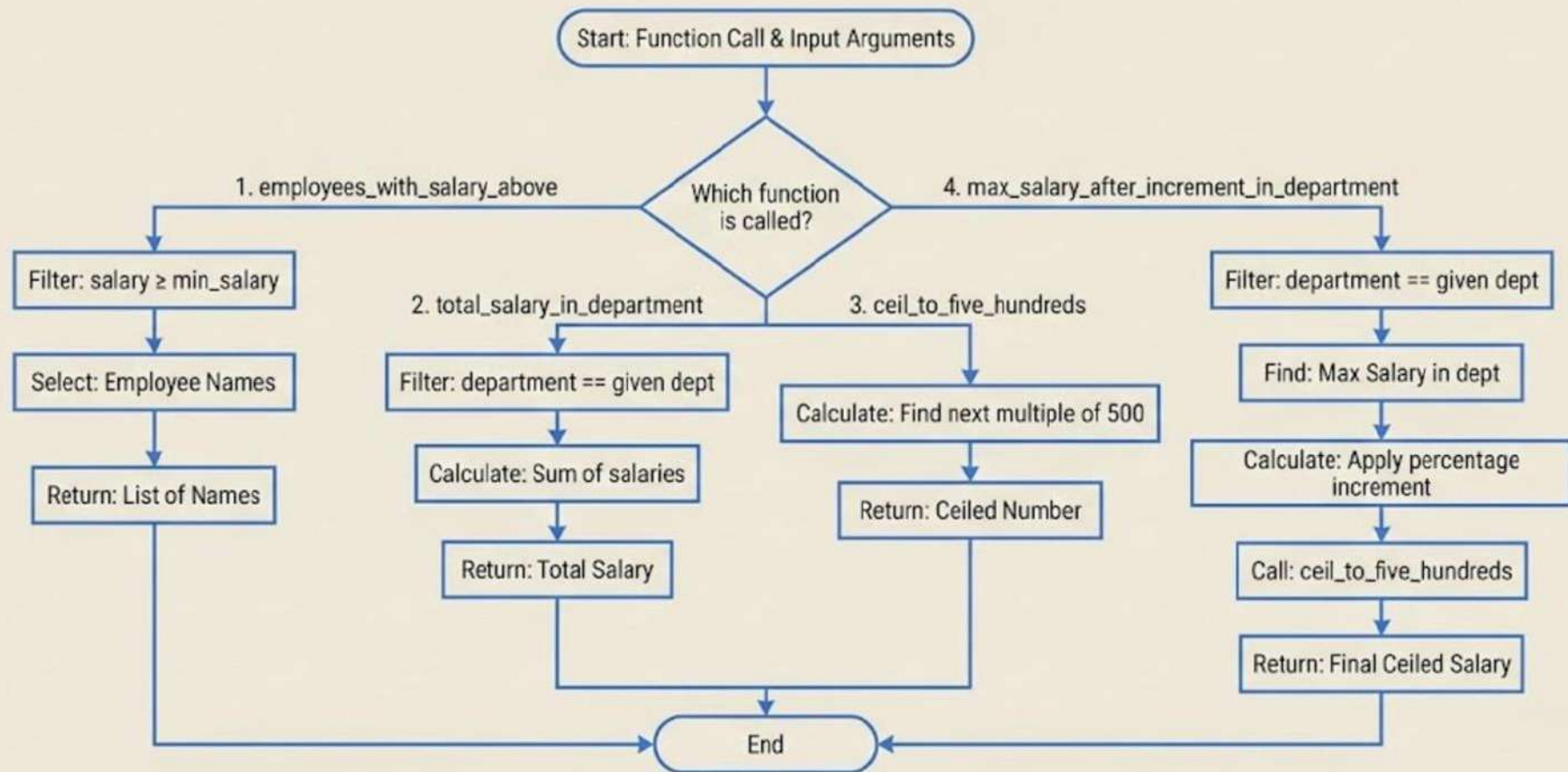
Visualizing Salary Filtering

Example Data: min_salary = 60000, employees = ['Alice', 'Bob', 'Charlie', 'David', 'Eve']



Result (Employees with Salary $\geq$ 60000) → ['Bob', 'David']

The Logic Flowchart



The Explanatory Approach

Step-by-step implementation for clarity.

1. Employees with salary above a minimum
def employees_with_salary_above(employees, min_salary):

```
    names = []  
    for emp in employees:  
        if emp['salary'] >= min_salary: # Check if salary meets the minimum  
            names.append(emp['name']) # Add name to list if condition met  
    return names # Return the list of names
```

Condition to filter employees based on salary threshold.

2. Total salary in a specific department
def total_salary_in_department(employees, department):

```
    total = 0  
    for emp in employees:  
        if emp['department'] == department: # Filter by department  
            total += emp['salary'] # Accumulate salary for the department  
    return total # Return the total salary
```

Condition to select employees from the target department.

Summing up the salaries for the selected department.

3. Ceil a number to the next five hundred
def ceil_to_five_hundreds(num):

```
    # Formula to round up to nearest 500  
    return ((num + 499) // 500) * 500
```

Mathematical formula to implement ceiling to the next 500.

4. Max salary after increment, ceiled

def max_salary_after_increment_in_department(employees, department, inc_percent):

```
    department_salaries = []  
    for emp in employees:  
        if emp['department'] == department: # Filter by department  
            department_salaries.append(emp['salary'])  
    max_salary = max(department_salaries) # Find max salary in department  
    increment_amount = max_salary * (inc_percent / 100) # Calculate increment  
    incremented_salary = max_salary + increment_amount # Add increment  
    # Ceil the final incremented salary  
    return ceil_to_five_hundreds(int(incremented_salary))
```

Identifying the highest salary within the department.

Calculating the new salary after applying the percentage increase.

Applying the ceiling function to the final incremented salary.

The Pythonic Optimization

leveraging built-ins for concise logic.

List comprehensions, generator expressions, `sum()`, and `max()` for concise logic.

Hinting
priority. →

```
def employees_with_salary_above(employees: list[dict], min_salary: int) -> list[str]:  
    return [emp['name'] for emp in employees if emp['salary'] >= min_salary]  
  
def total_salary_in_department(employees: list[dict], department: str) -> int:  
    return sum(emp['salary'] for emp in employees if emp['department'] == department)  
  
def ceil_to_five_hundreds(num: int) -> int:  
    return ((num + 499) // 500) * 500  
  
def max_salary_after_increment_in_department(employees: list[dict], department: str, inc_percent: int) -> int:  
    return ((max(emp['salary'] for emp in employees if emp['department'] == department) * (1 +  
        inc_percent / 100) + 499) // 500) * 500
```

Single-line executions reduce
verbosity and improve readability.

Result: Identical output, cleaner syntax

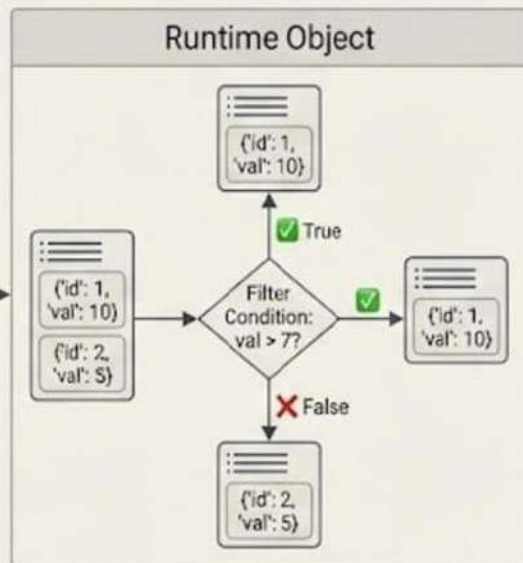
Concept Spotlight: Filtering a List of Dictionaries

Iterate through records and select items based on a specific condition.

To process data stored as a list of dictionaries, you iterate through the list using a loop. For each dictionary, you access its values via keys (e.g., `item['key']`) and use a conditional statement (like `if`) to decide whether to include it in your result or perform an action.

Code View

```
data = [{'id': 1, 'val': 10}, {'id': 2, 'val': 5}]  
filtered = [d for d in data if d['val'] > 7]  
# Result: [{'id': 1, 'val': 10}]
```



Application: Input: `employees = [{'name': 'Alice', 'salary': 50000}, {'name': 'Bob', 'salary': 70000}]`
-> Process: Filter for `'salary' > 60000` -> Output: `['Bob']` (Names only)

Concept Spotlight: Ceiling to Nearest Multiple

Rounding a number upward to the next required multiple using integer arithmetic.

EXPLANATION

Sometimes we must **always round upward**, even if the number is not close to the next multiple.

This operation is called **ceiling**.

To round a number up to the next multiple of 500, we use this formula:

$$((\text{num} + 499) // 500) * 500$$

Why it works:

- **+499 ensures** any number that is not already a multiple of 500 moves into the 'next' group.
- **// 500** performs integer division to determine which 500-block the number belongs to.
- **+ 500** converts that block index back into the actual multiple of 500.

This method avoids floating point math and guarantees the result is always rounded upwards.

CODE VIEW

Ceiling to the next multiple of 500

```
result = ((num + 499) // 500) * 500
```

Ceiling to the next multiple of 500

```
result = ((num + 499) // 500) * 500
```

RUNTIME OBJECT

Example: num = 25100

Step 1 — Shift to next block
 $25100 + 499 = 25599$

Step 2 — Find the block index
 $25599 // 500 = 51$

Step 3 — Convert block to actual value
 $51 * 500 = 25500$

Final Result 25500

Application:

Input: 25100 → Output: 25500

Meaning:

The next valid multiple of 500 greater than or equal to 25100 is 25500.

Solution Comparison

Conceptual Approaches to Employee Data Analysis

Step-by-Step Approach



- High Readability.
- Clear intermediate steps.
- Modular Functions
- Best for: Beginners & Debugging.
Debugging.

Concise Optimization



- Pythonic Syntax.
- Efficient Use & Built-btins.
- Less Verbosity
- Best for: Production &
Advanced Users.

Result: Both achieve identical outcomes with different methoologies.

Key Takeaways

01

Filter Employees

Select employees whose salary is above a certain threshold.

02

Calculate Totals

Sum the salaries of employees within a specific department.

03

Round Numbers

Round up numbers to the nearest five hundred.

04

Max Salary Increment

Increase and round up the highest salary in a department by a given percentage.

Final Syntax Cheat Sheet

```
return [emp['name'] for emp in employees if emp['salary'] > min_salary]
return sum(emp['salary'] for emp in employees if emp['department'] == dept)
return ((num + 499) // 500) * 500
return ((max(emp['salary'] for emp in employees if emp['department'] == dept) * (1 +
inc_percent / 100) + 499) // 500) * 500
```